Wigell AutoCore 4.0 – Migrations- och riskanalys

Oct 19, 2026

Planen beskriver hur Grupp D flyttar Wigell AutoCore från Spring Boot 2.7 och Java 8 till Spring Boot 4.x och Java 21, och samtidigt inför inloggning, behörigheter, strukturerad loggning och ett fristående körläge. Den svarar på punkterna 1–8 i avsnitt 10 i kravspecifikationen 4.0. Planen är ett levande dokument: upptäcker teamet en ny betydande risk läggs den in under avsnitt 2, och ändringen noteras i ändringsloggen sist i dokumentet.

Utgångsläge: main och dev är identiska efter PR #136 (sprint WAC-3.5). Där finns 343 körda tester, 0 fel, och CI-checken `mvnw clean verify` körs på varje PR.

1. Nuvarande plattform och beroenden

Komponent	Nu	Mål	Kommentar
Java	8	21 (LTS)	Kravet är 17+. 21 är förvalt, 25 är möjligt
Spring Boot	2.7.18	4.x	Två major-hopp, görs stegvis via 3.x
Persistens-API	`javax.persistence`, `javax.validation`	`jakarta.*`	31 filer berörs
Hibernate	5 (via Boot 2.7)	7 (via Boot 4)	Striktare typmappning
SQLite-dialekt	`sqlite-dialect` 0.1.2	`hibernate-community-dialects`	Gamla dialekten stöder inte Hibernate 6+
SQLite JDBC	3.53.4.0	Senaste kompatibla	Verifieras
Flyway	`flyway-core`, migreringar V1–V13	`spring-boot-starter-flyway`	Utan startern körs migreringarna inte i Boot 4
JavaFX	Ingår i Liberica JDK 8 + FX	OpenJFX-beroenden i `pom.xml`	Ingår inte i JDK 17+
Test	JUnit 5.13.4 och Mockito 4.11.0, hårdlåsta	Versioner från Boots BOM	Mockito 4 fungerar dåligt på Java 17+
Validering	`spring-boot-starter-validation` låst till 2.7.18	Från BOM	Låsningen tas bort
CI	`setup-java` Liberica 8 `jdk+fx`	Java 21	`build-check.yml`
Loggning	SLF4J/Logback via Boot, ingen egen konfiguration	`logback-spring.xml` per profil	Se avsnitt 6
Säkerhet	Ingen	Spring Security 7	Se avsnitt 5

2. Identifierade risker

Den största risken är att två major-uppgraderingar samtidigt gör fel omöjliga att lokalisera. Därför görs plattformsbytet i två steg med gröna tester emellan.

3. Delar som behöver förändras

Affärsreglerna i `model` ändras inte i sak. Koden delas upp i tre Maven-moduler. `autocore-core` innehåller logiken och beror inte på Spring Boot eller JavaFX. `autocore-app` och `autocore-standalone` bygger på core.

4. Bevarande av befintlig information

Den nya versionen öppnar befintlig `autocore.db` utan att ta bort data.

- **Migreringar:** V1–V13 rörs inte, eftersom checksummorna måste stämma. Allt nytt läggs i V14 och framåt.
- **Fixturtest:** en kopia av `autocore.db` från WAC-3.5 ligger i `src/test/resources`. Testet kontrollerar att Flyway når V14 och att data går att läsa med oförändrade belopp och datum.
- **Schemavalidering:** `ddl-auto=validate` i test.
- **Demo:** demodatabasen säkerhetskopieras före första start.
- **Förbjudet:** att radera databasen för att komma runt ett migreringsfel.

5. Autentisering och behörighetsstyrning

Behörighet kontrolleras i servicelagret och inte i vyerna. Då finns ingen väg in som går förbi kontrollen. Spring Security används utan webbserver.

Behörighetsmatris

Operation	Admin	Verkstad	Läs
Lista och visa kunder, fordon, bokningar, arbetsorder, fakturor	Ja	Ja	Ja
Kunder och fordon: skapa och ändra	Ja	Ja	Nej
Kunder och fordon: ta bort	Ja	Nej	Nej
Bokningar: skapa, ändra, avboka	Ja	Ja	Nej
Arbetsorder: skapa, uppdatera, bekräfta, starta, slutföra, avbryta, återöppna	Ja	Ja	Nej
Fakturor: skapa	Ja	Nej	Nej
Betalningar: registrera	Ja	Nej	Nej
Priser, tjänster och tjänstepaket: ändra	Ja	Nej	Nej
Mekaniker: hantera	Ja	Nej	Nej

6. Loggning och felhantering

Varje användaroperation får ett korrelations-id. Samma id syns i feldialogen och på varje loggrad, så ett fel går att följa från klicket till stället där det uppstod.

- **Ramverk:** SLF4J med Logback, som följer med Spring Boot. `logback-spring.xml` har profiler för dev (konsol, DEBUG för `com.wac`), test och headless (fil, INFO). Det fristående läget startar inte Spring och får därför en egen `logback.xml`.
- **Format:** tid, nivå, logger (systemdel), korrelations-id, operation, användare och meddelande. Värdena läggs i MDC i början av varje operation och rensas efteråt.

- **Konsolutskrifter ersätts:** `System.err` och `printStackTrace` i `GlobalExceptionHandler`, `HomeView` och `AlertHelper` byts mot loggern. `HeadlessRunner` behåller `System.out` för sin rapport, eftersom det är programutdata.
- **Skydd av känslig information:** lösenord, hashar och tokens loggas aldrig. Entiteter loggas inte; id:n används i stället. `Customer.toString()` slutar innehålla namn, telefon och e-post.
- **Användaren ser:** förväntade fel visas som i dag, med språknyckel och text. Oväntade fel och databasfel visar ett allmänt meddelande med korrelations-id. Stack tracen finns bara i loggen.

Händelser i stället för loggrader i tjänsterna: `WorkOrderService` publicerar en `WorkOrderEvent` per statusändring (Observer). En lyssnare i `autocore-app` loggar händelsen först efter commit, så en återställd operation inte loggas. Inloggningar, misslyckade inloggningar och nekad åtkomst loggas via Spring Securitys egna händelser på samma sätt.

7. Fristående verksamhetsfunktionalitet

Modulen `autocore-standalone` kör två operationer med samma domän- och servicekod som appen, utan Spring, GUI eller webbserver.

Operationer

1. **Arbetsorderns livscykel:** skapa, bekräfta, starta och slutför en drop-in-order. Därefter två otillåtna fall: att starta en avbruten order, och att en läsbehörig användare försöker slutföra.
2. **Fakturering:** fakturan räknas ut med rabattstrategi, till exempel `WELCOME10`.

Uppbyggnad

- **Komposition:** tjänsterna skapas med `new` och kopplas mot in-memory-repositoryn.
- **Behörighet:** en dekoratör frågar samma `AccessPolicy` som `@PreAuthorize`, med rollen som programargument.
- **Begränsning:** Bean Validation och transaktioner körs inte utan Spring. Domänreglerna gäller fullt ut.
- **Paketering:** en körbar jar via `maven-shade-plugin`, med egen `logback.xml`.
- **Vakter:** enforcer-regler och ett arkitekturtest förbjuder Spring-kontext och JavaFX i modulen.

8. Kontroll av tidigare funktionalitet

Alla befintliga tester ska vara gröna efter varje migreringssteg, innan arbetet går vidare. Inget test tas bort utan att en ersättare finns.

- **Regressionssvit:** 343 körda tester (enhetstester, servicetester, integrationstester mot SQLite, migreringstester, arkitekturtest) körs i CI på Java 21.
- **Datatest:** testet mot WAC-3.5-fixturdatabasen från avsnitt 4.
- **Nya säkerhetstester:** för varje roll anropas en tillåten och en nekad operation direkt på servicen. Ett test kontrollerar att oinloggad och obehörig ger olika fel. Ett test kontrollerar att headless utan konto stoppas.
- **Loggtest:** ett test som fångar loggutdata kontrollerar att ett misslyckat inloggningsförsök inte skriver ut lösenordet.

- **Fristående läge:** programmet körs som test och avslutas med kod 0, och arkitekturtestet vaktar att Spring inte startas.
- **Manuell genomgång:** före demon går teamet igenom huvudflödena i GUI:t med alla tre kontona, det vill säga bokning, arbetsorder, faktura och betalning.

JavaFX-gränssnitt

Gränssnittet ändras bara där det behövs för konsekvent felrapportering och en enhetlig struktur. Ingen ny verksamhetsfunktionalitet tillkommer. Arbetet börjar efter moduluppdelningen, eftersom vyerna flyttar till `autocore-app`.

- **Gemensam dialogstruktur:** en abstrakt `FormDialog<T>` bygger alla formulärdialoger i samma ordning (Template Method). Dialogerna fyller bara i innehåll och sparlogik. I dag är de byggda på två olika sätt.
- **Fel visas i dialogen:** affärsfel visas i dialogen, som hålls öppen, med samma texter som i headless. I dag visas fel på tre olika sätt. Fakturering och betalning anropar dessutom tjänsten först när dialogen stängts, så användaren tappar det hen fyllt i om något går fel.
- **Sammanslagen faktureringsvy:** faktura- och betalningsvyerna blir en vy med en fakturadialog. Kräver beställarens godkännande först (se risktabellen).
- **Uppdatera CSS:** färger samlas som variabler i `.root` i `style.css`, och inga `setStyle`-anrop finns kvar i koden.